COMPLETE_EXAMPLES_TUTORIALS

HelixCode Examples & Tutorials

Overview

This guide provides practical examples and step-by-step tutorials for using HelixCode in various development scenarios. From basic setup to advanced enterprise deployments, these examples demonstrate real-world usage patterns and best practices.

Quick Start Examples

Basic AI Code Generation

Scenario: Generate a Go function to reverse a string

```
# Start HelixCode
helix start

# Generate code using local LLM
helix generate "Write a Go function that reverses a string
        efficiently"

# Output:
func reverseString(s string) string {
    runes := []rune(s)
    for i, j := 0, len(runes)-1; i < j; i, j = i+1, j-1 {
        runes[i], runes[j] = runes[j], runes[i]
    }
    return string(runes)
}
```

Multi-Provider Code Review

Scenario: Get code review feedback from multiple AI providers

```
# Configure multiple providers
helix-config set llm.providers.openai.api_key "sk-your-key"
helix-config set llm.providers.anthropic.api_key "sk-ant-your-
        key"

# Generate and review code
helix generate "Create a REST API in Go with proper error
        handling" --model claude-3-sonnet
```

```
# Get review from different provider
helix generate "Review this Go code for security issues and
        performance" --model gpt-4 --context-file api.go
```

Tutorial: Building a Web API

Step 1: Project Setup

```
# Create new project directory
mkdir my-web-api && cd my-web-api

# Initialize Go module
go mod init my-web-api

# Create basic project structure
mkdir -p cmd/server internal/{handlers,middleware,models}
```

Step 2: Generate API Structure

```
# Generate main server file
helix generate "Create a main.go file for a Go web server using
        Gin framework"

# Generate user model
helix generate
        "Create a User struct with JSON tags for id, name, email,
        created_at"

# Generate API handlers
helix generate "Create REST API handlers for CRUD operations on
        users"
```

Step 3: Add Authentication

```
# Generate JWT authentication middleware
helix generate "Create JWT authentication middleware for Gin"

# Generate login/register endpoints
helix generate "Create login and register HTTP handlers with JWT
        tokens"

# Add password hashing
helix generate "Add bcrypt password hashing to user registration"
```

Step 4: Database Integration

```
# Generate database models
helix generate "Create PostgreSQL database schema for users
        table"

# Generate database connection
helix generate "Create database connection and migration
        functions"

# Generate repository layer
helix generate "Create user repository with CRUD operations"
```

Step 5: Testing & Validation

```
# Generate unit tests
helix generate "Create unit tests for user handlers"

# Generate integration tests
helix generate "Create integration tests for user API endpoints"

# Run tests
go test ./...
```

Tutorial: Distributed Computing Setup

Step 1: Configure Worker Pool

```
# Add worker nodes
helix worker add worker1.example.com --user helix --key ~/.ssh/
        id_rsa

# List workers
helix worker list

# Check worker status
helix worker status
```

Step 2: Configure Task Distribution

```
# config/config.yaml
workers:
  health_check_interval: 30
  max_concurrent_tasks: 10
  auto_scaling: true
  min_workers: 2
  max_workers: 10
```

```yaml
tasks:
  max_retries: 3
  checkpoint_interval: 300
  timeout: 3600
```

Step 3: Submit Distributed Tasks

```
# Submit code generation task
helix task submit --type code-generation --prompt "Create a
        microservice in Go"

# Submit testing task
helix task submit --type testing --target ./my-project

# Monitor task progress
helix task list
helix task status <task-id>
```

Step 4: Monitor Performance

```
# Check worker utilization
curl http://localhost:8080/metrics | grep worker

# View task queue
helix task queue

# Get performance report
helix performance report
```

Tutorial: Enterprise Security Setup

Step 1: Configure Security Scanning

```yaml
# config/config.yaml
security:
  scanning:
    enabled: true
    zero_tolerance: true
    schedule: "0 */4 * * *"
  headers:
    enabled: true
    csp: "default-src 'self'"
    hsts: true
```

Step 2: Set Up Authentication

```
# Configure JWT
export HELIX_AUTH_JWT_SECRET=$(openssl rand -hex 32)
```

```
# Set up RBAC
helix-config set auth.rbac.enabled true

# Create admin user
helix user create admin --role admin
```

Step 3: Configure Audit Logging

```
# config/config.yaml
logging:
  level: info
  audit:
    enabled: true
    file: "/var/log/helixcode/audit.log"
    format: json
```

Step 4: Run Security Assessment

```
# Run security scan
./bin/helixcode-security-fix --scan

# Check security status
helix security status

# View security report
cat reports/security/latest-report.md
```

Tutorial: Performance Optimization

Step 1: Establish Baselines

```
# Create performance baseline
helix performance baseline create

# Run load test
helix performance test --duration 5m --concurrency 100

# View baseline metrics
helix performance baseline show
```

Step 2: Apply Optimizations

```
# Run automated optimization
helix performance optimize

# Configure specific optimizations
helix-config set performance.cpu_optimization true
```

```
helix-config set performance.memory_optimization true
helix-config set performance.cache_optimization true
```

Step 3: Monitor Improvements

```
# Compare against baseline
helix performance baseline compare

# View optimization report
cat reports/performance/optimization-report.txt

# Set up continuous monitoring
helix performance monitor --continuous
```

Tutorial: Multi-Provider LLM Setup

Step 1: Configure Providers

```
# OpenAI (premium)
helix-config set llm.providers.openai.api_key "sk-your-key"
helix-config set llm.providers.openai.models '["gpt-4", "gpt-3.5-
        turbo"]'

# Anthropic (premium)
helix-config set llm.providers.anthropic.api_key "sk-ant-your-
        key"
helix-config set llm.providers.anthropic.models '["claude-3-
        sonnet", "claude-3-haiku"]'

# Free providers (no API keys needed)
helix-config set llm.providers.xai.enabled true
helix-config set llm.providers.openrouter.enabled true
```

Step 2: Set Up Fallback Strategy

```
# config/config.yaml
llm:
  default_provider: "anthropic"
  fallback_providers:
    - "openai"
    - "xai"
    - "openrouter"
  retry_attempts: 3
  retry_delay: "1s"
```

Step 3: Test Provider Switching

```
# Test with primary provider
helix generate "Explain quantum computing" --model claude-3-
        sonnet

# Test fallback (simulate primary failure)
helix generate "Write a Python function" --fallback

# Check provider status
helix llm status
```

Tutorial: CI/CD Integration

Step 1: GitHub Actions Setup

```
# .github/workflows/helixcode.yml
name: HelixCode CI/CD

on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Set up Go
        uses: actions/setup-go@v4
        with:
          go-version: '1.21'

      - name: Run tests
        run: make test

      - name: Security scan
        run: make security-scan

      - name: Performance test
        run: make performance-test
```

Step 2: Docker Integration

```
# Dockerfile
FROM golang:1.26-alpine AS builder

WORKDIR /app
COPY go.mod go.sum ./
```

```
RUN go mod download

COPY . .
RUN make build

FROM alpine:3.18
RUN apk add --no-cache ca-certificates
COPY --from=builder /app/bin/helixcode /usr/local/bin/

EXPOSE 8080
CMD ["helixcode", "server"]
```

Step 3: Kubernetes Deployment

```yaml
# k8s/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: helixcode
spec:
  replicas: 3
  selector:
    matchLabels:
      app: helixcode
  template:
    metadata:
      labels:
        app: helixcode
    spec:
      containers:
      - name: helixcode
        image: helixcode:latest
        ports:
        - containerPort: 8080
        env:
        - name: HELIX_DATABASE_PASSWORD
          valueFrom:
            secretKeyRef:
              name: helixcode-secrets
              key: db-password
        resources:
          requests:
            cpu: 500m
            memory: 1Gi
          limits:
            cpu: 2000m
            memory: 4Gi
```

Step 4: Monitoring Integration

```
# Deploy with monitoring
kubectl apply -f k8s/

# Check deployment status
kubectl get pods -l app=helixcode

# View logs
kubectl logs -f deployment/helixcode

# Set up monitoring
kubectl apply -f k8s/monitoring/
```

Tutorial: Custom Tool Development

Step 1: Create Tool Structure

```
// internal/tools/custom/example.go
package custom

import (
    "context"
    "dev.helix.code/internal/tools"
)

type ExampleTool struct{}

func (t *ExampleTool) Name() string {
    return "example"
}

func (t *ExampleTool) Description() string {
    return "Example custom tool for demonstration"
}

func (t *ExampleTool) Execute(ctx context.Context, params
        map[string]interface{}) (map[string]interface{}, error) {
    message := params["message"].(string)

    return map[string]interface{}{
        "result": "Hello, " + message,
        "timestamp": time.Now(),
    }, nil
}
```

Step 2: Register Tool

```
// internal/tools/registry.go
func init() {
    RegisterTool(&custom.ExampleTool{})
}
```

Step 3: Configure Tool

```
# config/config.yaml
tools:
  custom:
    example:
      enabled: true
      timeout: 30s
```

Step 4: Test Tool

```
# Test custom tool
helix tool run example --message "World"

# Output: {"result": "Hello, World", "timestamp":
        "2024-01-20T10:00:00Z"}
```

Tutorial: Advanced Context Management

Step 1: Set Up Context Sessions

```
# Create development session
helix session create dev-session --description "Web API
        development"

# Switch to session
helix session switch dev-session
```

Step 2: Add Context Items

```
# Add file to context
helix context add main.go

# Add project documentation
helix context add README.md docs/

# Add search results
helix context search "authentication" --add
```

Step 3: Use Context in Generation

```
# Generate code with context awareness
helix generate "Add authentication middleware to the API" --use-
        context

# The AI will have access to:
# - Current codebase structure
# - Existing authentication patterns
# - Project documentation
# - Recent changes
```

Step 4: Manage Context Sessions

```
# List context items
helix context list

# Remove outdated items
helix context remove old-file.go

# Save session for later
helix session save dev-session

# Load session in new terminal
helix session load dev-session
```

Tutorial: Notification System Setup

Step 1: Configure Notification Channels

```
# config/config.yaml
notifications:
  slack:
    webhook_url: "${HELIX_SLACK_WEBHOOK_URL}"
    channel: "#devops"
  email:
    smtp_server: "smtp.gmail.com"
    smtp_port: 587
    username: "${HELIX_EMAIL_USERNAME}"
    password: "${HELIX_EMAIL_PASSWORD}"
  discord:
    webhook_url: "${HELIX_DISCORD_WEBHOOK_URL}"
```

Step 2: Set Up Alerts

```
# Configure performance alerts
helix alert create high-cpu --condition "cpu_usage > 80" --
        severity critical --channels slack,email
```

```
# Configure security alerts
helix alert create security-scan-failed --condition
        "security_scan_status == 'failed'" --severity high --
        channels slack

# Configure system alerts
helix alert create disk-space-low --condition
        "disk_usage_percent > 85" --severity warning --channels
        email
```

Step 3: Test Notifications

```
# Send test notification
helix notify "System startup completed successfully" --type
        success

# Trigger alert manually
helix alert trigger high-cpu --message "CPU usage at 95%"

# Check alert status
helix alert list
```

Tutorial: Backup & Recovery

Step 1: Configure Backup

```
# config/config.yaml
backup:
  enabled: true
  schedule: "0 2 * * *"  # Daily at 2 AM
  retention: "30d"
  compression: true
  encryption: true
  destinations:
    - type: "s3"
      bucket: "helixcode-backups"
      region: "us-east-1"
    - type: "local"
      path: "/var/backups/helixcode"
```

Step 2: Create Backup

```
# Manual backup
helix backup create --name manual-backup-2024

# List backups
helix backup list
```

```
# Verify backup integrity
helix backup verify manual-backup-2024
```

Step 3: Test Recovery

```
# Dry run recovery
helix backup restore manual-backup-2024 --dry-run

# Perform actual recovery
helix backup restore manual-backup-2024

# Verify recovery
helix health check
```

Tutorial: Multi-Environment Deployment

Step 1: Environment Configuration

```
# config/environments/development.yaml
environment: development
logging:
  level: debug
database:
  host: localhost
  debug: true
performance:
  optimization: false

# config/environments/production.yaml
environment: production
logging:
  level: info
database:
  host: postgres-prod
  sslmode: require
performance:
  optimization: true
security:
  scanning:
    enabled: true
    zero_tolerance: true
```

Step 2: Environment Switching

```
# Switch to development
helix env switch development
```

```
# Switch to production
helix env switch production

# Show current environment
helix env current
```

Step 3: Environment-Specific Commands

```
# Development workflow
helix env switch development
helix generate "Add debug logging" --model llama-3-8b

# Production deployment
helix env switch production
helix deploy --environment production
helix security scan
```

Best Practices Examples

Code Quality Standards

```
# Run comprehensive checks
make lint
make test
make security-scan

# Generate coverage report
make coverage

# Performance testing
make performance-test
```

Error Handling Patterns

```
// HelixCode-recommended error handling
func processRequest(ctx context.Context, req *Request)
        (*Response, error) {
    // Validate input
    if err := validateRequest(req); err != nil {
        return nil, fmt.Errorf("invalid request: %w", err)
    }

    // Process with timeout
    result, err := processWithTimeout(ctx, req)
    if err != nil {
        // Log error with context
        log.WithError(err).WithField("request_id",
        req.ID).Error("processing failed")
```

```
        // Return structured error
        return nil, &ProcessingError{
            Code:    "PROCESSING_FAILED",
            Message: "Failed to process request",
            Cause:   err,
        }
    }

    return result, nil
}
```

Testing Patterns

```
// Unit test example
func TestUserValidation(t *testing.T) {
    tests := []struct {
        name     string
        user     User
        wantErr  bool
        errField string
    }{
        {
            name: "valid user",
            user: User{Name: "John", Email: "john@example.com"},
            wantErr: false,
        },
        {
            name: "missing name",
            user: User{Email: "john@example.com"},
            wantErr:  true,
            errField: "name",
        },
    }

    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            err := validateUser(tt.user)
            if tt.wantErr {
                assert.Error(t, err)
                assert.Contains(t, err.Error(), tt.errField)
            } else {
                assert.NoError(t, err)
            }
        })
    }
}
```

Performance Monitoring

```
# Set up performance monitoring
helix performance monitor --continuous --interval 30s

# Create performance budget
helix performance budget set --response-time 200ms --cpu 70% --
        memory 1GB

# Run performance regression tests
helix performance test --compare-baseline
```

This examples and tutorials guide provides practical, hands-on guidance for using HelixCode in various development scenarios. Each tutorial builds on real-world use cases and demonstrates best practices for enterprise AI development. docs/COMPLETE_EXAMPLES_TUTORIALS.md ## Sources verified Sources verified 2026-05-29: https://go.dev/doc/devel/release (Go 1.26.3 confirmed latest stable; Dockerfile base image corrected golang:1.21-alpine → golang:1.26-alpine to match inner go.mod go 1.26 / CLAUDE.md §3.1) ; https://www.postgresql.org/support/versioning/ (PostgreSQL 15 supported, latest minor 15.18) ; https://github.com/redis/redis/releases (Redis 7+ valid; latest GA 8.8.0) — tutorial build/deploy version references cross-checked against official sources. Project version authority is go.mod + CLAUDE.md §3.1.